

Sliding Window Technique

From Brute Force to Optimized Solutions

> "Whenever I see a contiguous subarray or substring, my first thought should be **Sliding Window**."

DATA STRUCTURES & ALGORITHMS

2-HOUR LECTURE

Learning Outcomes

By the end of this lecture, you will be able to:

1

Identify

Recognise Sliding Window problems from keywords and patterns

2

Fixed Window

Understand and implement fixed-size window algorithms

3

Variable Window

Handle dynamically resizing windows for flexible constraints

4

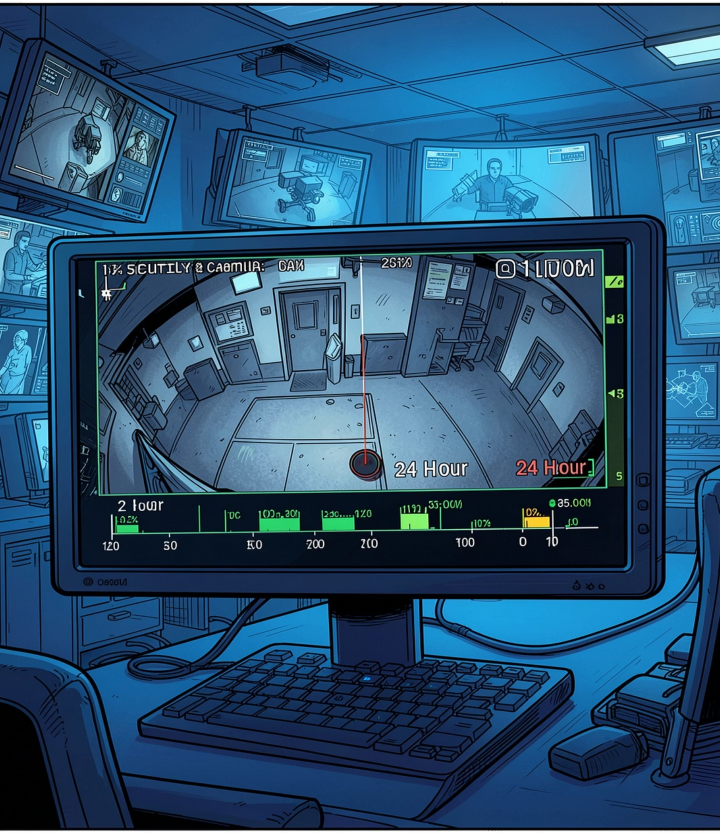
Optimise

Convert $O(n^2)$ brute force solutions into $O(n)$

5

Solve

Tackle real interview questions with confidence



Warm-Up: The CCTV Problem

You have **24 hours of CCTV footage**. Management asks: *"Find the busiest continuous 1-hour duration."*

❌ Brute Force

Count every hour from scratch – repeated work

✅ Sliding Window

Reuse the previous count – subtract old, add new

Key insight: Instead of re-observing the whole world, you only update what changed. This is Sliding Window.

Real-Life Analogies



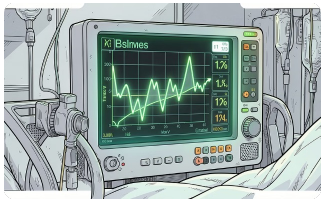
🚂 Train Window

The window size never changes. As the train moves, scenes exit left and enter right. You only notice what *changed* – not the whole world again.



🛒 Shopping Cart Discount

Maximum discount on every **consecutive 3 items**. Instead of recalculating each group, reuse the previous sum – subtract the item that left, add the new one.



🌡️ Temperature Monitoring

Highest average of the **last 5 minutes**. Every new reading: remove the oldest, add the newest. Window size = 5, always.

Why Sliding Window? The Math

❌ Brute Force – O(n·k)

Array: `[1, 4, 2, 8, 5, 9, 3]`, k = 3

- Window 1: 1+4+2 = **7**
- Window 2: 4+2+8 = **14** ← recalculates 4+2
- Window 3: 2+8+5 = **15** ← recalculates 2+8

Repeated work at every step.

✅ Sliding Window – O(n)

Array: `[1, 4, 2, 8, 5, 9, 3]`, k = 3

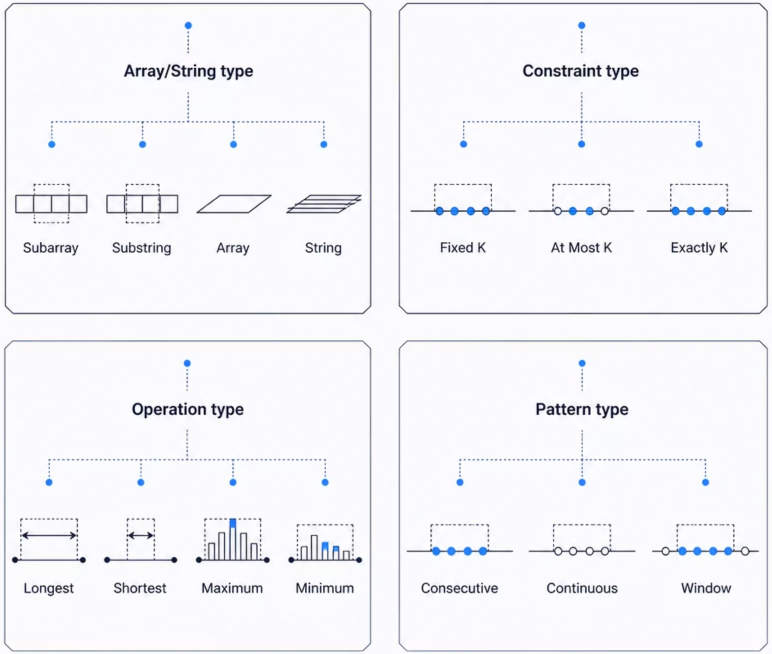
- Window 1: sum = **7**
- Window 2: 7 − 1 + 8 = **14**
- Window 3: 14 − 4 + 5 = **15**

Formula: newSum = oldSum − left + right

When to Think Sliding Window

Train yourself to spot these **trigger keywords** in any problem statement. If you see two or more, Sliding Window is almost certainly the right approach.

Interview Trick: If the question mentions "continuous," "consecutive," "subarray," or "k elements" – think Sliding Window immediately.



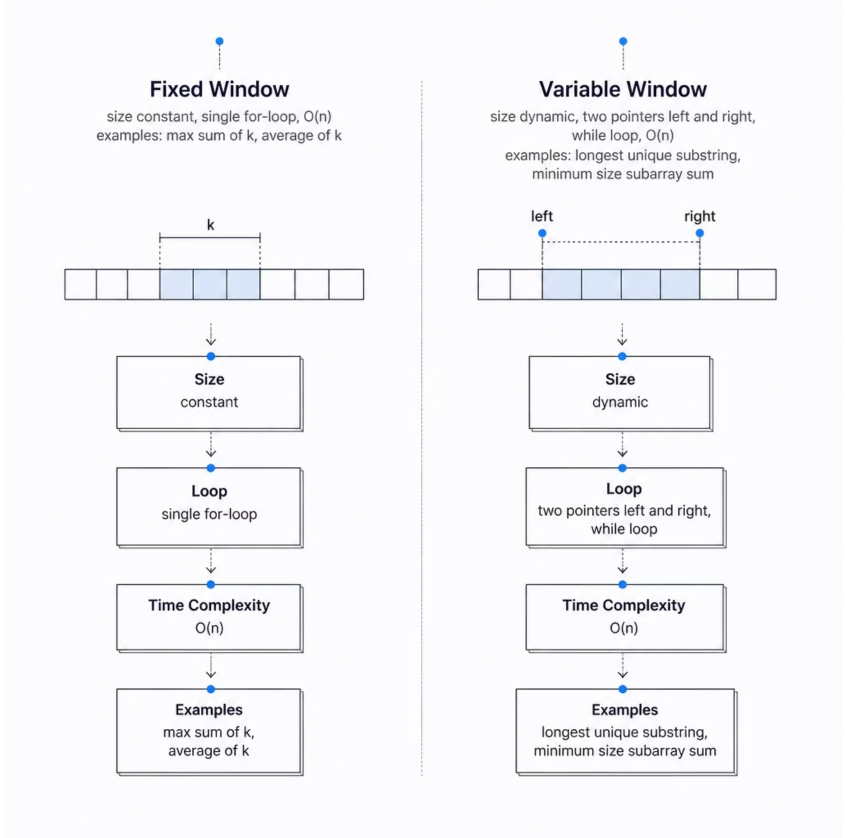
Two Types of Sliding Window

Fixed Window

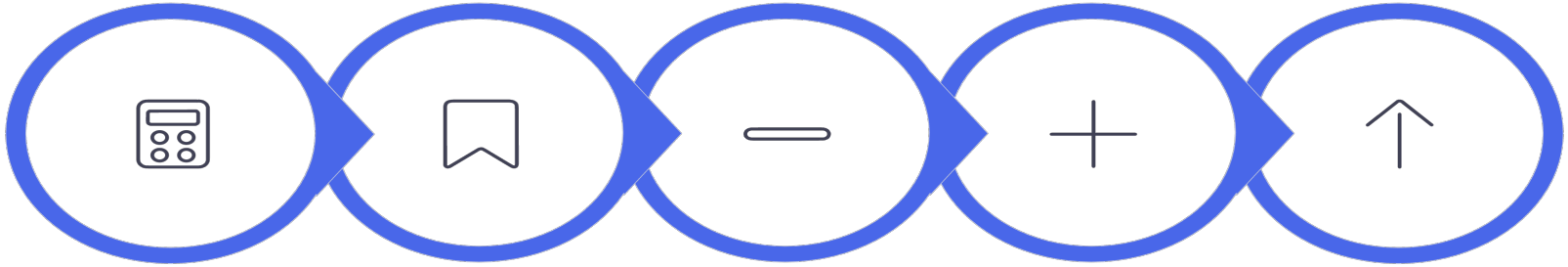
Window size is predetermined (k). Slides one step at a time. **Example:** Maximum sum of k consecutive elements.

Variable Window

Window expands and shrinks dynamically based on a condition. **Example:** Longest substring without repeating characters.



Algorithm: Fixed Window



Each step involves only two arithmetic operations – making the entire pass **O(n)**.

Dry Run – arr = [2, 5, 1, 8, 2, 9, 1], k = 3

Left	Right	Window	Sum	Max
0	2	[2,5,1]	8	8
1	3	[5,1,8]	14	14
2	4	[1,8,2]	11	14
3	5	[8,2,9]	19	19

Code Snippet (Python)

```
def maxSum(arr, k):
 n = len(arr)
 curr = sum(arr[:k])
 maxs = curr
 for i in range(k, n):
 curr += arr[i] - arr[i-k]
 maxs = max(maxs, curr)
 return maxs
```

Practice & Interview Problems

Easy

- Max Sum Subarray of Size K
- Average of All Subarrays of Size K
- First Negative in Every Window

Medium

- Longest Substring Without Repeating Characters
- Minimum Size Subarray Sum
- Longest Repeating Character Replacement

Hard

- Find All Anagrams in a String
- Minimum Window Substring
- Count Subarrays With Fixed Bounds

Pattern Flowchart: Array/String → Continuous/Consecutive? → Subarray/Substring? → Max/Min/Longest/Shortest? → **Sliding Window**



Key Takeaways

1 **Spot the Pattern**

Keywords like "consecutive," "subarray," and "k elements" signal Sliding Window.

2 **Fixed vs Variable**

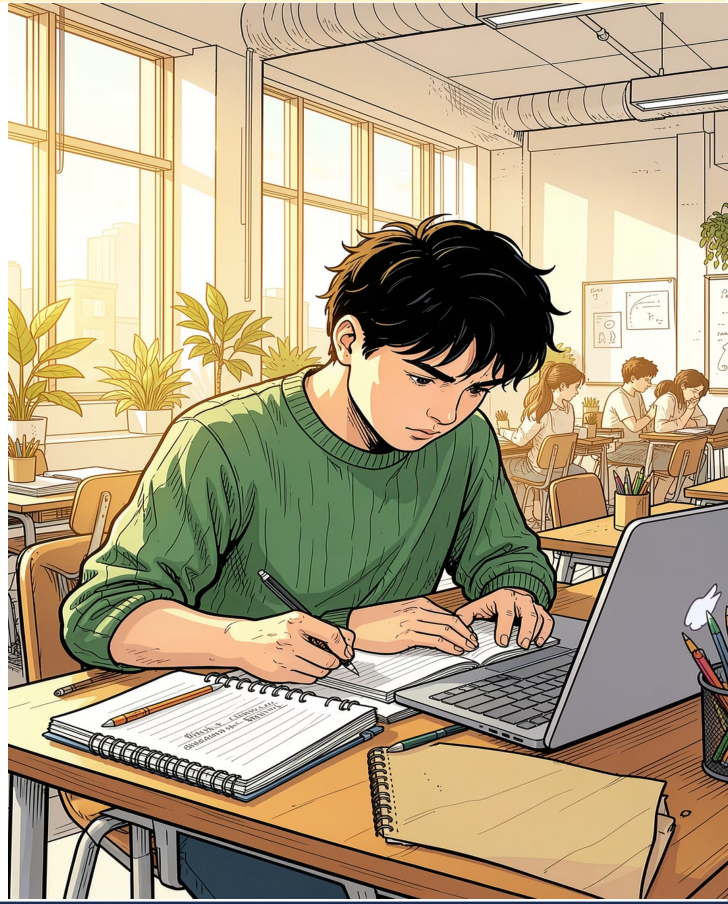
Fixed size = simple loop. Variable size = two pointers that expand and shrink.

3 **Optimise Smartly**

Reuse previous results: `newSum = oldSum - left + right`

4 **Master the Template**

Initialise → Expand → Check condition → Shrink → Update result. Practise daily!



Sliding Window Templates (Pseudo Code)

1. Fixed Sliding Window

```
left = 0
right = 0

while (right < n)

    // Include current element

    if (window size == k)

        // Process current window
        // Update answer

        // Remove left element

        left++

    right++
```

2. Variable Sliding Window

```
left = 0
right = 0

while (right < n)

    // Include current element

    while (window is invalid)

        // Remove left element

        left++

    // Window is valid
    // Update answer

    right++
```

3. Variable Sliding Window (Minimum Window Problems)

```
left = 0
right = 0

while (right < n)

    // Include current element

    while (window satisfies condition)

        // Update minimum answer

        // Remove left element

        left++

    right++
```

Easy Rule to Remember

Fixed Window

```
Add
↓
Window Size == K
↓
Process
↓
Remove
↓
Slide
```

Variable Window

```
Add
↓
Invalid?
↓
Yes → Remove Until Valid
↓
Update Answer
↓
Expand
```

Example 01: Maximum Sum Subarray of Size

```
left = 0
right = 0
sum = 0
maxSum = −∞

while (right < n)

    // Include current element
    sum = sum + arr[right]

    if (window size == k)

        // Process current window
        maxSum = max(maxSum, sum)

        // Remove left element
        sum = sum − arr[left]

        left++

    right++

return maxSum
```

Generic Pattern Mapping

Generic Template	This Problem
Include current element	`sum += arr[right]`
Process current window	`maxSum = max(maxSum, sum)`
Remove left element	`sum -= arr[left]`
Answer	`maxSum`

Example 02: First Negative Integer in Every Window of Size K.

```
left = 0
right = 0

Store all negative numbers (or their indices)

while (right < n)

    // Include current element
    if (arr[right] < 0)
        Add it to the list/queue

    if (window size == k)

        // Process current window
        if (list/queue is empty)
            answer = 0
        else
            answer = first negative number

        // Remove left element
        if (arr[left] is the first negative)
            Remove it from the list/queue

        left++

    right++
```

Generic Pattern Mapping

Generic Template	This Problem
Include current element	If `arr[right] < 0`, add it to the queue/list
Process current window	Output the first negative number, or `0` if none exists
Remove left element	If `arr[left]` is the first negative, remove it from the queue/list
Answer	List of first negative numbers for each window